

Procesos de software

Cuando hablamos de la ingeniería de *Software*, debemos tener en claro que tenemos diferentes procesos para utilizar en el desarrollo de un *software*. Cada modelo con sus características correspondientes.

Hay que tener en cuenta que, ante la decisión de un cambio, es necesario tener presentes los argumentos necesarios para validar el cambio y, también, poder identificar las metodologías que se pueden utilizar en el desarrollo del *software*.

Procesos de software

Referencias

Descarga en PDF

Procesos de software

La empresa Sumar se encuentra en un proceso de cambio muy amplio en el área de sistemas y se encuentra buscando diferentes consultoras que la asesoren para poder desarrollar los cambios previstos en diferentes áreas de la empresa y poder seleccionar los procesos a utilizar para realizar cambios en el sistema de cobranzas y el sistema de clientes.

Hay proyectos en el área de ventas como el sistema de comisiones a vendedores que son cortos y tienen muy en claro los requerimientos funcionales y no funcionales.

Ante la situación planteada, la empresa Sumar requiere asesoramiento para poder desarrollar cada uno de los proyectos.

Para poder aportar soluciones a la empresa Sumar, se le pueden detallar los procesos que puede utilizar en el desarrollo del *software*.

Modelo de procesos

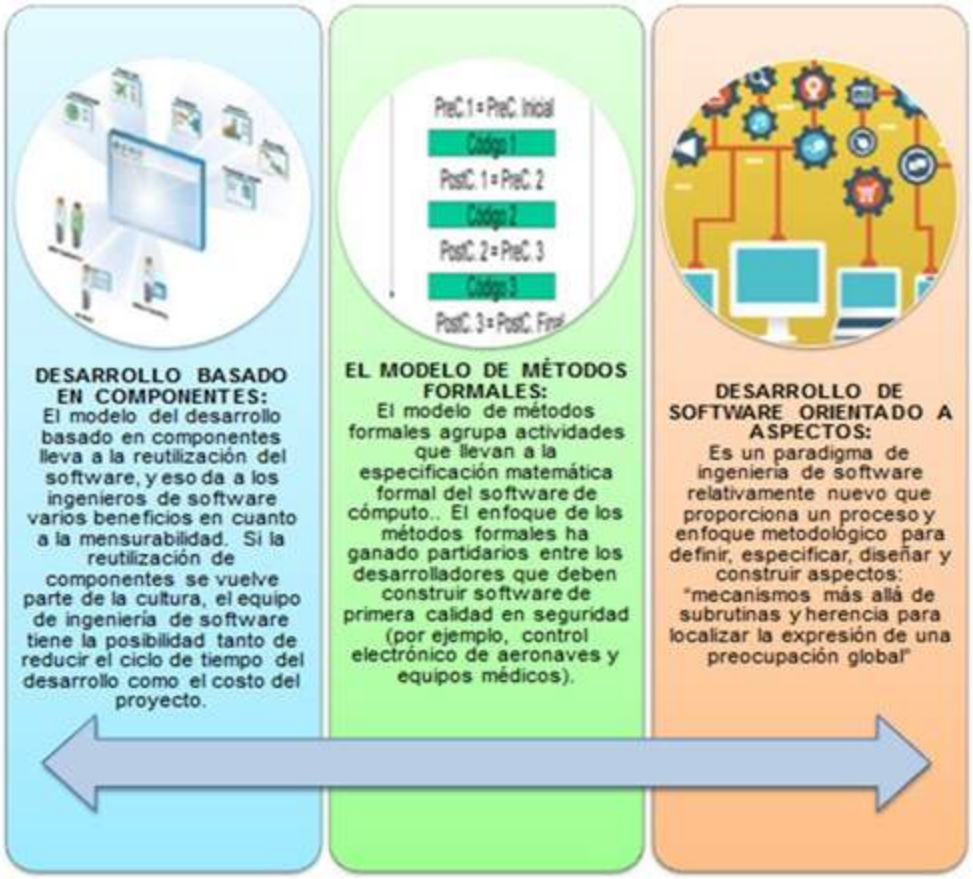
Un modelo de proceso es un conjunto de actividades, tareas y acciones que se realizan con el fin de alcanzar el desarrollo completo de un proyecto de *software*.

Existen diferentes modelos de proceso como:

1. **Modelo de proceso especializado**

   Los modelos de proceso especializado tienen muchas de las características de uno o más de los modelos tradicionales que se presentaron en las secciones anteriores. Sin embargo, dichos modelos tienden a aplicarse cuando se elige un enfoque de ingeniería de *software* especializado o definido muy específicamente.

Figura 1: Los modelos de proceso especializados



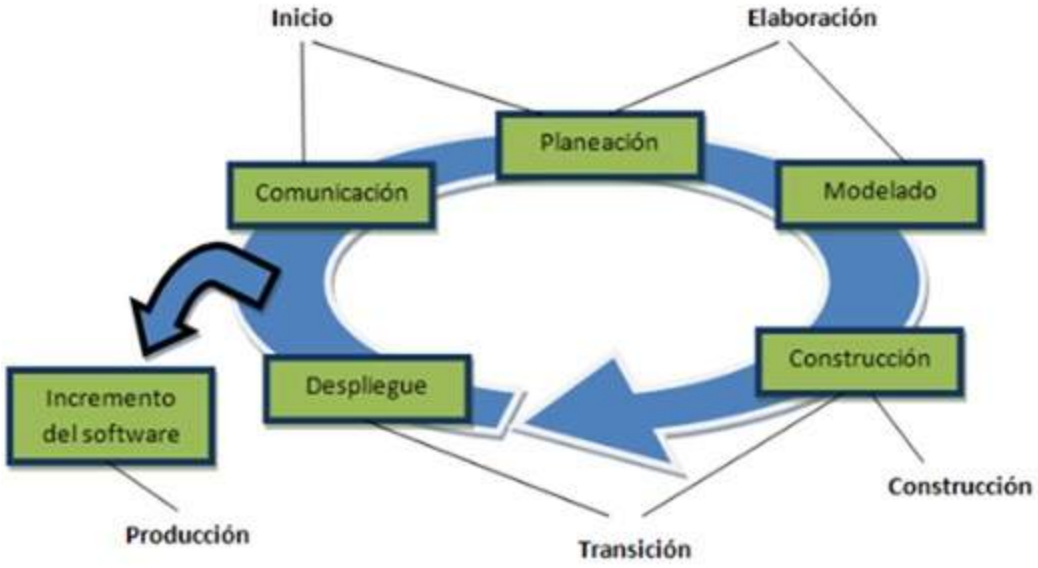
Fuente: [imagen sin título sobre Los modelos de proceso especializado], 2015, https://bit.ly/3yLAkgn

---

2 El proceso unificado

El proceso unificado es un intento por obtener los mejores rasgos y características de los modelos tradicionales del proceso del *software*, pero en forma que implemente muchos de los mejores principios del desarrollo ágil de *software*. El proceso unificado reconoce la importancia de la comunicación con el cliente y los métodos directos para describir su punto de vista respecto de un sistema. Sugiere un flujo del proceso iterativo e incremental, lo que da la sensación evolutiva que resulta esencial en el desarrollo moderno del *software*.

Figura 2: Figura modelo del proceso unificado



Fuente: [imagen sin título sobre el proceso unificado], 2015, https://bit.ly/3yLAkgn

La fase de concepción del proceso unificado agrupa actividades tanto de comunicación con el cliente como de planeación. Al colaborar con los participantes, se identifican los requerimientos del negocio, se propone una arquitectura aproximada para el sistema y se desarrolla un plan para la naturaleza iterativa e incremental del proyecto en cuestión.

3 Proceso personal del *software*

El proceso personal del *software* pone el énfasis en la medición personal tanto del producto del trabajo que se genera como de su calidad. Además, el PPS responsabiliza al profesional acerca de la planeación del proyecto (por ejemplo, estimación y programación

de actividades) y delega en el practicante el poder de controlar la calidad de todos los productos del trabajo de *software* que se desarrollen. El proceso personal del *software* define cinco actividades estructurales.

Figura 3: Figura modelo del proceso personal del *software*



Fuente: [imagen sin título sobre el proceso personal del software], 2015, https://bit.ly/3yLAkgn

4 Proceso del equipo de *software*

El objetivo de este proceso es construir un equipo autodirigido para el proyecto, que se organice para producir un *software* de alta calidad.

Los objetivos que se definen para el proceso del equipo de software son los siguientes:

- Formar equipos autodirigidos que planeen y den seguimiento a

su trabajo, que establezcan metas y que sean dueños de sus procesos y planes.

- Mostrar a los gerentes cómo dirigir y motivar a sus equipos y cómo ayudarlos a mantener un rendimiento máximo.
- Acelerar la mejora del proceso del *software*, haciendo del modelo de madurez de la capacidad, CMM, 23 nivel 5, el comportamiento normal y esperado.
- Brindar a las organizaciones muy maduras una guía para la mejora.
- Facilitar la enseñanza universitaria de aptitudes de equipo con grado industrial.

El proceso del equipo de *software* define las siguientes actividades estructurales: inicio del proyecto, diseño de alto nivel, implementación, integración y pruebas y *post mortem*. Como contrapartes del proceso del equipo de *software*, estas actividades permiten que el equipo planee, diseñe y construya *software* en forma disciplinada, al mismo tiempo que mide cuantitativamente el proceso y el producto. La etapa *post mortem* es el escenario de las mejoras del proceso.

Para Pressman (2006), el marco genérico del proceso comprende lo siguiente:

COMUNICACIÓN:	PLANEACIÓN:

Implica una intensa colaboración y comunicación con los clientes, que abarque la investigación de requisitos y otras actividades relacionadas.

COMUNICACIÓN:

PLANEACIÓN:

Establece un plan de trabajo de la ingeniería del ***software***.

Este describe las tareas técnicas que deben realizarse; los riesgos probables; los recursos que serán requeridos; los productos del trabajo que han de producirse y un programa de trabajo.

MODELADO:

CONSTRUCCIÓN:

DESPLIEGUE:

Abarca la creación de modelos que le permiten tanto al desarrollador como al cliente entender mejor los requisitos del ***software*** y el diseño que logrará satisfacerlos.

MODELADO:	CONSTRUCCIÓN:	DESPLIEGUE:

Esta actividad combina la generación del código y la realización de pruebas necesarias para descubrir errores en el código.

MODELADO:	CONSTRUCCIÓN:	DESPLIEGUE:

El ***software*** se entrega al cliente, quien evalúa el producto recibido y proporciona información basada en su evaluación.

Estas cinco actividades genéricas del marco trabajo son útiles durante:

- el desarrollo de programas pequeños;
- la creación de grandes aplicaciones en la red; y
- la ingeniería de sistemas basados en computadoras grandes y complejas.

En la etapa de modelado, intervienen el análisis y el diseño.

El análisis comprende la investigación, elaboración, negociación, especificación y validación de requerimientos.

El diseño comprende tareas de diseño de datos, diseño arquitectónico, diseño de interfaz y diseño a nivel de componentes.

La visión de Sommerville (2005)

Esta aporta que un modelo de procesos del *software* es una descripción simplificada de un proceso del *software* que presenta una visión de ese proceso.

Estos modelos pueden incluir actividades que son parte de los procesos y productos de *software* y el papel de las personas involucradas en la ingeniería del *software*. Algunos ejemplos de estos tipos de modelos que se pueden producir son los siguientes:

1. Un modelo de flujo de trabajo: muestra la secuencia de actividades en el proceso junto con sus entradas, salidas y dependencias. Las actividades, en este modelo, representan acciones humanas.

2. Un modelo de flujo de datos o de actividad: representa el proceso como un conjunto de actividades, cada una de las cuales realiza alguna transformación en los datos.

Muestra cómo la entrada en el proceso, tal como una especificación, se transforma en una salida, tal como un diseño. Pueden representar transformaciones llevadas a cabo por las personas o por las computadoras.

3. Un modelo de rol/acción: representa los roles de las personas involucradas en el proceso del *software* y las actividades de las que son responsables.

La mayor parte de los modelos de procesos del *software* están basados en uno de los tres modelos generales o paradigmas de desarrollo de *software*.

1. El enfoque en cascada: considera las actividades anteriores y las representa como fases de procesos separados, tales como la especificación de requerimientos, el diseño del *software*, la implementación, las pruebas, etcétera. Después de que cada etapa queda definida, "se firma" y el desarrollo continúa con la siguiente etapa.

2. Desarrollo iterativo: este enfoque entrelaza las actividades de especificación, desarrollo y validación. Un sistema inicial se desarrolla rápidamente a partir de especificaciones muy abstractas. Este se refina basándose en las peticiones del cliente para producir un sistema que satisfaga las necesidades de dicho cliente. El sistema puede, entonces, ser entregado. De forma alternativa, se puede volver a implementar utilizando un enfoque más estructurado para producir un sistema más sólido y mantenible.

3. Ingeniería del *software* basada en componentes: esta técnica supone que las partes del sistema existen. El proceso de desarrollo del sistema se enfoca en la integración de estas partes más que en desarrollarlas desde el principio.

Para desarrollar el asesoramiento en la gestión del cambio que necesita la empresa Sumar, se pueden establecer los siguientes elementos a tener en cuenta:

Gestión del cambio en ingeniería del *software*

El cambio es inherente al *software* computacional y genera confusión entre los ingenieros de *software* involucrados en un proyecto, dicha confusión surge cuando los cambios no se analizan antes de realizarlos, no se registran antes de implementarlos, no se reporta a quienes deben saberlo o no se mantiene un control sobre ellos.

La gestión del cambio es una actividad que se desarrolla durante todo el proceso de desarrollo, ya que no sabemos en el momento que se originará un cambio, las actividades en este proceso se desarrollan para:

1. identificar el cambio;
2. controlar el cambio;
3. garantizar que el cambio se realice de manera adecuada;
4. reportar los cambios a todos los interesados.

¿Quién es el encargado de la gestión del cambio?

Todos los involucrados en el desarrollo de un proyecto deberían, en alguna medida, participar en la gestión de cambio, aunque hay empresas con más recursos que tienen personas especializadas para este fin.

¿Por qué es importante?

Si el cambio no se controla en un proyecto de ***software***, este tiende al caos, más aún cuando hay una gran cantidad de personas involucradas y en posibles partes distintas, tanto así que un desarrollo de ***software*** sin control puede llegar a entregar productos de muy mala calidad, por lo cual es una práctica sólida de ingeniería de ***software***.

¿Cuáles son los pasos a seguir?

Primero, se deben identificar los productos de trabajo. Segundo, se deben establecer mecanismos para el control de versiones y cambio. Por último, se

debe auditar el proceso para asegurarse que la calidad se mantenga en el cambio y que los interesados reciban la información requerida.

¿Cuál es el producto obtenido?

Un plan de gestión de cambio.

¿Cómo estamos seguros de que se ha hecho bien?

Cuando cualquier producto puede explicarse, seguirse y controlarse y los cambios pueden seguirse y analizarse y todos los interesados están enterados del cambio que se ha hecho.

Existen cuatro fuentes fundamentales del cambio:

a. nuevas condiciones en el negocio;
b. nuevas necesidades del cliente;
c. reorganización o crecimiento del negocio;
d. restricciones presupuestales.

Elementos de un sistema de gestión de cambio

1. Elementos de componentes: herramienta que permite el acceso y gestión de cada elemento de gestión de cambio como, por ejemplo, la base de datos.
2. Elementos de proceso: serie de procedimientos y tareas que definen un enfoque eficaz con el cual gestionar el cambio.
3. Elementos de construcción: conjunto de herramientas que automatizan la construcción del *software* al asegurar que se ha ensamblado un conjunto de componentes válidos de *software*.
4. Elementos humanos: donde el equipo utiliza las herramientas y procesos para la gestión de cambio.

Etapas del proceso gestión de cambio

1. Identificación: se deben nombrar cada uno de los elementos que intervienen mediante un enfoque orientado a objetos.

2. Control de la versión: combina procedimientos y herramientas para gestionar diferentes versiones de objetos que se crean durante el proceso del *software*.

3. Control de cambio: se deben crear procesos para generar cambios al sistema, donde se debe evaluar el impacto y otros aspectos.

4. Auditoría de la configuración

Ayuda a asegurar que el cambio se ha realizado con propiedad, de acuerdo con las siguientes preguntas.

¿Se ha realizado el cambio específico? ¿Se han incorporado modificaciones adicionales? ¿Se ha realizado una revisión técnica formal para evaluar la corrección técnica? ¿Se ha seguido el proceso de *software*? ¿Se han aplicado debidamente los estándares de ingeniería de *software*? ¿Se han regido los procesos de gestión de cambio?

5. Reporte, informe de estado donde se resuelven las preguntas

¿Qué ocurrió? ¿Quién lo hizo? ¿Cuándo ocurrió? ¿Qué otra cosa será afectada?

Para garantizar una gestión de cambio exitosa, se deben tener en cuenta las siguientes características del software.

1. **Mantenibilidad**
   Debemos escribir *software* de tal forma que pueda evolucionar para cumplir las necesidades del cliente. El cambio en el *software* es una consecuencia inevitable.

Figura 4: Cableado extremo



Fuente: [imagen sin título sobre cableados], 2020, https://bit.ly/3hTFwbQ

¿Intentarías agregar una conexión nueva?

2 **Confiabilidad**

La confiabilidad del *software* se transmite en fiabilidad, protección y seguridad.
El *software* no debe causar daños físicos ni económicos en caso de una falla del sistema.

3 **Eficiencia**

El *software* debe ser diseñado de manera que no desgaste los recursos del sistema como memoria o procesador (tiempos de respuesta, tiempo de procesamiento, etc.).

Figura 5: Diseño eficiente



Fuente: [imagen sin título sobre centro de datos de Google], s. f., https://bit.ly/3yEOuj2

4

Usabilidad

El *software* que diseñamos debe ser de fácil uso para el usuario final que fue diseñado, no debería generar esfuerzo adicional (interfaz y documentación simples).

Al tener en claro los requerimientos funcionales y no funcionales, el área de ventas de la empresa le puede informar la utilización de la metodología RUP.

Haremos un repaso de los principales puntos de la lectura.

Para Pressman (2006), la planeación establece un diseño del *software*.

- Verdadero.
- Falso.

SUBMIT

Según Sommerville (2005), existe un modelo de flujo de trabajo. Identifique qué desarrolla el modelo propuesto:

- Muestra la secuencia de actividades en el proceso junto con sus entradas, salidas y dependencias.
- Muestra la secuencia de interfaz en el proceso junto con sus entradas, salidas y dependencias.
- Muestra la secuencia de caso de uso en el proceso junto con sus entradas, salidas y dependencias.
- Muestra la secuencia de actividades de diseño en el proceso junto con sus entradas, salidas y dependencias.

- ◯ Muestra la secuencia de actividades junto con sus entradas, salidas y dependencias.

SUBMIT

Identificar qué atributos se deben cumplir en un *software* para una gestión de cambio exitosa:

- ☐ Usabilidad .
- ☐ Mantenibilidad.
- ☐ Confiabilidad.
- ☐ Confidencialidad.
- ☐ Disponibilidad.

SUBMIT

Hay un atributo en el cual el *software* que diseñamos debe ser de fácil uso para el usuario final que fue diseñado, no debería generar esfuerzo adicional. Identifica en la siguiente lista el atributo correspondiente:

- ◯ Usabilidad.
- ◯ Mantenibilidad.
- ◯ Confidencialidad.
- ◯ Disponibilidad.

SUBMIT

CONTINUAR

Referencias

[Imagen sin título sobre Los modelos de proceso especializados]. (2015). Recuperado de https://ingsotfwarekarlacevallos.wordpress.com/category/modelos-de-proceso/

[Imagen sin título sobre el proceso unificado]. (2015). Recuperado de https://ingsotfwarekarlacevallos.wordpress.com/category/modelos-de-proceso/

[Imagen sin título sobre el proceso personal del software]. (2015). Recuperado de https://ingsotfwarekarlacevallos.wordpress.com/category/modelos-de-proceso/

[Imagen sin título sobre cableados]. (2020). Recuperado de https://afinidadelectrica.com/wp-content/uploads/2020/05/art222-cableados-trafo2.jpg

[Imagen sin título sobre centro de datos de Google]. (s. f.). Recuperado de https://static.dw.com/image/16313211_403.jpg

Pressman, R. S. (2006). *Ingeniería De Software: Un Enfoque Práctico*. México: Editorial McGraw Hill.

Sommerville, I. (2005). *Ingeniería de Software*. Madrid, España: Pearson Educación S. A.

CONTINUAR

Descarga en PDF

PDF

Módulo 1 - Lectura 2.pdf

1.2 MB